Types of Docker Networks

In this train, we'll explore the different types of networks created by Docker, such as bridge, host, and none. We'll discuss their characteristics and use cases, enhancing your ability to select the appropriate network type for your applications.

Learning objectives

- By the end of this train, you should be able to recognise the different types of networks created by Docker and their use cases.
- Understand how each Docker network type functions internally.
- Select appropriate network types based on application requirements and scenarios.

Bridge Network

The **bridge** network is Docker's default network type. It creates an isolated internal network on the host machine, allowing containers to communicate with each other.

```
# Example: Creating and using a bridge network
$ docker network create --driver bridge my-bridge-network
$ docker run -d --name container1 --network my-bridge-network nginx
$ docker run -d --name container2 --network my-bridge-network alpine sleep 10
00
```

Use case: Perfect for multi-container applications on the same host where internal communication is required, such as a web server and database communicating privately.

Host Network

The **host** network removes isolation between the container and the host, meaning the container shares the host's network stack.

```
# Example: Running a container with host network
$ docker run --network host nginx
```

Use case: Ideal for high-performance applications like monitoring agents that require low latency and direct access to host ports and interfaces.

None Network

The **none** network disables networking completely for the container.

```
# Example: Running a container with no network access
$ docker run --network none alpine
```

Use case: Best suited for containers performing tasks that do not require network communication, providing a security benefit by isolating them from any external access.

Overlay Network

The **overlay** network allows containers running on different Docker hosts to communicate securely. It is commonly used in Docker Swarm clusters.

```
# Example: Creating an overlay network
$ docker network create --driver overlay my-overlay-network
```

Use case: Useful for microservice architectures deployed across multiple hosts, where containers need to communicate securely across nodes.

Macvlan Network

The **macvlan** network driver assigns MAC addresses to containers, making them appear as physical devices on the network.

```
# Example: Creating a macvlan network
$ docker network create -d macvlan \
    --subnet=192.168.1.0/24 \
    --gateway=192.168.1.1 \
    -o parent=eth0 my-macvlan-network
```

Use case: Ideal for legacy applications or systems that require direct connectivity on the physical network, such as network appliances or older systems expecting unique MAC addresses.

Custom Network Plugins

Docker also supports third-party custom network plugins that can provide advanced networking functionalities such as security policies, encrypted networks, and integration with SDNs.

Use case: Enterprise-level environments requiring complex networking setups beyond the default Docker drivers.

Conclusion

Docker networking offers flexibility and power through various network drivers. Understanding the characteristics and use cases of each type enables you to design efficient, secure, and scalable containerised applications tailored to your specific needs.

Key Takeaways

- **Takeaway 1:** The bridge network is Docker's default for communication between containers on the same host.
- **Takeaway 2:** Host networks provide high performance by sharing the host's network stack.
- **Takeaway 3:** None networks isolate the container completely, disabling networking.
- **Takeaway 4:** Overlay networks are crucial for multi-host communication in Docker Swarm clusters.
- **Takeaway 5:** Macvlan networks integrate containers directly into physical networks, essential for legacy or specialised systems.